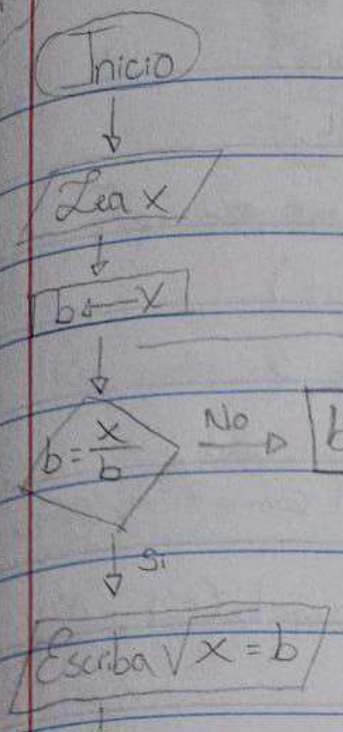


$x = 2$

$b = 2$

$2 = ? = \frac{2}{2}$

$2 = ? = 1$

F → $b = 0.5 * (\frac{2}{2} * 2)$

$b = 05 * 3 = 1.5$

◇ $1.5 = ? = \frac{2}{1.5} = 1.3333$

F

$\sqrt{(2)} = 1.4142$

$b = 0.5 (\frac{2}{1.3333} + 1.3333)$

$b = 0.5 * 2.8333 = 1.4166$

◇ $b = ? = \frac{2}{1.4166}$

$b = 0.5 (\frac{2}{1.4166} + 1.4166)$

$b = 0.5 * 1.4118 + 1.4118$

$0.5 * 2.8284 = 1.4142$

◇ $b = 1.4142 = ? = \frac{2}{1.4142}$

$\sqrt{2} = 1.4142$

Comandos Git

1. Init 2. Commit 3. Push 4. Branch 5. Pull

- Git init → hicia un repositorio Git. (Git init)
- Git commit → Guarda los cambios en Git (Git add .
  Git commit -m "primer commit")
- Git push → Sube los cambios al repositorio remoto (Git push origin main)
- Git branch → Muestra o crea ramas (Git branch)
  (Git branch nombre-de-la-rama)
- Git pull → Descarga los cambios del repositorio remoto
  (Git pull origin main)
- Git add → añadir archivos
- Git rm

Fundamentos de Programación

Piensa en C

Capítulo I

Keywords & Key concepts

- Algoritmo
- Problema
- Diagrama de flujo
- Datos
- Variables
- Constantes
- Identificadores
- Operadores
- Expresiones
- Programa en C

Notes

Algoritmos:

Un algoritmo es un conjunto ordenado de pasos que permite resolver un problema o alcanzar un resultado. Para resolver un problema se analiza la situación, se construye el algoritmo y luego se verifica que funcione correctamente. Un algoritmo debe ser preciso, determinista y finito.

Diagramas de flujo:

Representan gráficamente un algoritmo. Utilizan símbolos para indicar entrada de datos, procesos, decisiones y salida de resultados. El flujo normalmente se organiza de arriba hacia abajo y de izqui. a derecha.

Tipos de datos:
Los datos simples incluyen principalmente enteros, reales y caracteres. Las constantes mantienen su valor durante la ejecución, mientras que las variables pueden cambiar.

Operadores:
Permiten realizar operaciones. Hay operadores aritméticos, relacionales y lógicos. También existen operadores de incremento, decremento y asignación.

Programas en C:
Un programa es un conjunto de instrucciones que la computadora ejecuta para obtener un resultado. En C, primero se diseña la solución mediante un algoritmo o diagrama de flujo y después se transforma en código.

Actions / Nex steps
- ☐ Identificar el problema
- ☐ Determinar los datos de entrada y salida
- ☐ Crear el algoritmo
- ☐ Representarlo con un diagrama de flujo
- ☐ Convertirlo en un programa C

Questions & Reflections

- Cómo se transforma un problema en un algoritmo?
- Qué diferencia existe entre una variable y una constante?
- Para qué sirve el diagrama de flujo?
- Qué tipo de operador necesito según la operación?

Summary

El capítulo explica las bases para resolver problemas mediante programación. Primero se analiza el problema y se diseña un algoritmo formado por pasos precisos, determinados y finitos. Después, el algoritmo puede representarse mediante un diagrama de flujo, usando símbolos que muestran procesos, decisiones, entradas y salidas. También se presentan los tipos de datos, variables, constantes e identificadores, además de los principales operadores. Finalmente, se explica cómo llevar la solución a un programa en C, siguiendo una estructura ordenada.

Key Takeaway

Antes de programar hay que aprender a resolver el problema. Analizar → Diseñar el algoritmo → Representar la solución → Programarla en C.

Matemáticas para la Computación

Capítulos I y II

Keywords & Key concepts

- Sistemas numéricos
- Decimal, binario, octal y hexadecimal
- Conversión de bases
- Complemento a 2
- Algoritmo de Booth
- Métodos de conteo
- Principio del producto y adición
- Permutaciones
- Combinaciones
- Principio del palomar

Notes

Cap. 1 - Sistemas numéricos:

Los sistemas numéricos permiten representar cantidades usando diferentes bases. Los más utilizados en computación son decimal (10), binario (2), octal (8) y hexadecimal (16). Una cantidad puede convertirse de una base a otra mediante procedimientos específicos. En cada sistema, el valor de un número depende de la posición de sus dígitos.

Las operaciones básicas son suma, resta, multiplicación y división. En computación se utiliza el complemento a 2 para representar números negativos y realizar restas mediante sumas. El algoritmo de Booth permite realizar multiplicaciones binarias, incluyendo números con signo.

Cap. 2 - Métodos de conteo:

Los métodos de conteo permiten determinar cuántas posibilidades existen sin tener que enumerarlas una por una. El principio del producto multiplica las posibilidades de etapas sucesivas: $n \times m$. El principio de la adición suma posibilidades cuando las opciones son alternativas: $n + m$.

Las permutaciones consideran el orden de los elementos. Las combinaciones no consideran el orden. El factorial se representa como $*n! = n(n-1)...1.*$ El principio del palomar permite demostrar que, al distribuir más objetos que lugares disponibles, al menos un lugar tendrá más de un objeto.

Actions / Next steps

- ☐ Identificar la base numérica.
- ☐ Practicar conversiones entre bases.
- ☐ Realizar operaciones binarias
- ☐ Diferenciar permutación y combinación
- ☐ Aplicar producto o adición según el problema

Questions & Reflections

- Cómo convierto un número de una base a otra?
- Para qué sirve el complemento a 2?
- Cuando uso el principio del producto o de la adición?
- Cuál es la diferencia entre permutación y combinación?

Summary

Los sistemas numéricos permiten representar y procesar información en diferentes bases, especialmente en decimal, binario, octal y hexadecimal. Las conversiones y operaciones entre bases son fundamentales en computación. El completo a 2 y el algoritmo de Booth permiten trabajar con números binarios y operaciones con signo. Por otra parte, los métodos de conteo permiten calcular posibilidades mediante los principios de producto y adición, además de permutaciones y combinaciones.

Key Takeaway

En computación, los sistemas numéricos permiten representar información y los métodos de conteo permiten calcular de forma organizada las posibilidades de un problema.